

108. 将有序数组转换为二叉搜索树

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Easy	(Not Available)	(Not Available)	-	-	0

- **Tags:** 树, 二叉搜索树, 数组, 分治, 二叉树
- **Companies:** (Not Available)

给你一个整数数组 `nums`，其中元素已经按 **升序** 排列，请你将其转换为一棵 **高度平衡** 二叉搜索树。

高度平衡 二叉树是一棵满足「每个节点的左右两个子树的高度差的绝对值不超过 1」的二叉树。

示例 1:

输入：nums = [-10,-3,0,5,9]

输出：[0,-3,9,-10,null,5]

解释：[0,-10,5,null,-3,null,9] 也将被视为正确答案：

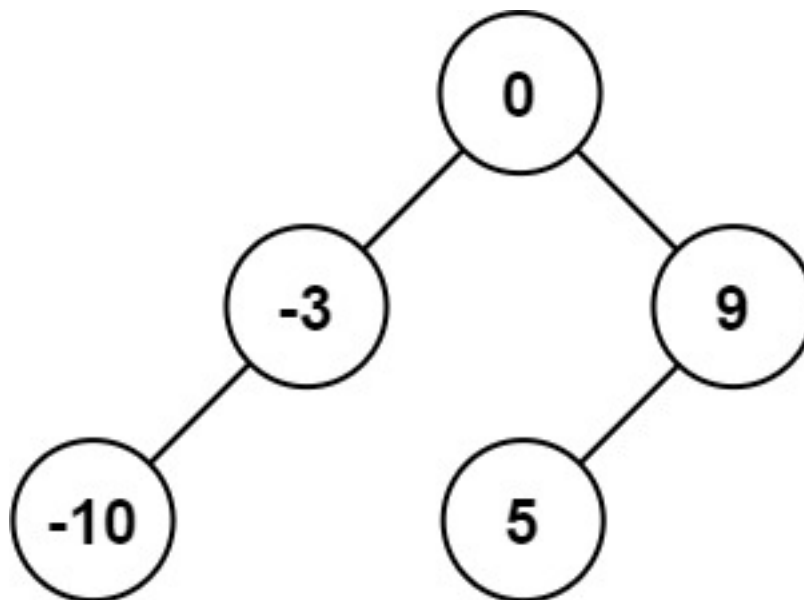


Figure 1: img

示例 2:

输入：nums = [1,3]

输出：[3,1]

解释：[1,null,3] 和 [3,1] 都是高度平衡二叉搜索树。

提示:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `nums` 按 **严格递增** 顺序排列

Discussion | Solution

解决方法

1. 问题理解

目标: 将一个升序排列的有序数组转换为一棵高度平衡的二叉搜索树 (BST)。**二叉搜索树 (BST)**

性质: - 左子树所有节点值 < 根节点值 < 右子树所有节点值。- 左右子树也分别是 BST。

高度平衡性质: - 每个节点的左右子树高度差不超过 1。**输入:** 升序数组 `nums`。**输出:** 高度平衡 BST 的根节点。

2. 思路分析

递归 (分治) 要构建一棵 BST, 我们需要选择一个根节点, 然后递归地构建左右子树。对于有序数组, 选择中间元素作为根节点可以天然地满足 BST 的性质 (左边都小, 右边都大), 并且这种选择方式最有利于构造**高度平衡**的树。

1. 定义递归函数 `build(left, right)`:

- **功能:** 使用数组 `nums` 中索引从 `left` 到 `right` (包含 `left` 和 `right`) 的元素构建一个高度平衡的 BST 子树, 并返回该子树的根节点。
- **基本情况 (Base Case):** 如果 `left > right`, 表示当前区间为空, 无法构建节点, 返回 `null`。
- **递归步骤 (Recursive Step):**
 - **选择根节点:** 计算中间索引 `mid = left + (right - left) // 2` (这种写法可以防止 `left + right` 溢出, 虽然 Python 中整数不会溢出, 但这是个好习惯)。创建根节点 `root = TreeNode(nums[mid])`。
 - **构建左子树:** 递归调用 `build(left, mid - 1)`, 将返回的子树根节点赋值给 `root.left`。左子树使用 `mid` 左侧的元素构建。
 - **构建右子树:** 递归调用 `build(mid + 1, right)`, 将返回的子树根节点赋值给 `root.right`。右子树使用 `mid` 右侧的元素构建。
 - **返回根节点:** 返回当前构建的子树根节点 `root`。

2. 主函数调用:

- 调用 `build(0, len(nums) - 1)` 开始构建整棵树。

为什么这样能保证高度平衡? 每次都选择中间元素作为根, 使得左右子树分配到的元素数量尽可能相等 (最多差 1)。这意味着左右子树的高度也会尽可能接近, 从而保证了整棵树的高度平衡。

执行流程示例 (`nums = [-10, -3, 0, 5, 9]`):

1. `build(0, 4) -> mid = 2, root = TreeNode(0)`

```

2. -> root.left = build(0, 1) -> mid = 0, left_root = TreeNode(-10)
3. -> left_root.left = build(0, -1) -> 返回 null
4. -> left_root.right = build(1, 1) -> mid = 1, right_sub_root =
    TreeNode(-3)
5. -> right_sub_root.left = build(1, 0) -> 返回 null
6. -> right_sub_root.right = build(2, 1) -> 返回 null
7. -> 返回 right_sub_root (TreeNode(-3))
8. -> left_root.right 设为 TreeNode(-3)。返回 left_root (TreeNode(-10))
9. -> root.left 设为 TreeNode(-10)。
10. -> root.right = build(3, 4) -> mid = 3, right_root = TreeNode(5)
11. -> right_root.left = build(3, 2) -> 返回 null
12. -> right_root.right = build(4, 4) -> mid = 4, right_sub_root =
    TreeNode(9)
13. -> right_sub_root.left = build(4, 3) -> 返回 null
14. -> right_sub_root.right = build(5, 4) -> 返回 null
15. -> 返回 right_sub_root (TreeNode(9))
16. -> right_root.right 设为 TreeNode(9)。返回 right_root (TreeNode(5))
17. -> root.right 设为 TreeNode(5)。
18. 返回 root (TreeNode(0))。

```

最终树结构: [0, -10, 5, null, -3, null, 9] (或选择 mid = (left + right + 1) // 2 得到 [0, -3, 9, -10, null, 5])

3. 代码实现 (Python)

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

from typing import List, Optional

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:
        """
        将有序数组转换为高度平衡的二叉搜索树
        """

```

```

def build(left: int, right: int) -> Optional[TreeNode]:
    """
    使用 nums[left...right] 构建 BST 子树
    """
    # 基本情况: 区间无效
    if left > right:
        return None

    # 选择中间元素作为根节点
    # mid = (left + right) // 2 # 倾向于选择左中位数
    mid = left + (right - left) // 2 # 更健壮的写法, 倾向于选择左中位数
    # mid = (left + right + 1) // 2 # 倾向于选择右中位数
    # mid = left + (right - left + 1) // 2 # 倾向于选择右中位数

    root = TreeNode(nums[mid])

    # 递归构建左右子树
    root.left = build(left, mid - 1)
    root.right = build(mid + 1, right)

    return root

# 从整个数组范围开始构建
return build(0, len(nums) - 1)

```

4. 复杂度分析

- **时间复杂度:** $O(N)$, 其中 N 是数组 `nums` 的长度。我们需要遍历数组中的每个元素一次来创建对应的节点。
- **空间复杂度:** $O(\log N)$ 。递归调用栈的深度。由于我们总是选择中间元素作为根, 树的高度是 $O(\log N)$, 因此递归栈的最大深度也是 $O(\log N)$ 。如果考虑返回的树本身占用的空间, 则为 $O(N)$ 。

5. 如何在面试中讲解

1. 明确目标与约束:

- “目标是将一个升序数组转换为 BST, 并且要求这棵 BST 是高度平衡的。”
- “BST 的性质是左小右大, 高度平衡意味着左右子树高度差不超过 1。”

2. 核心思路 (分治/递归):

- “有序数组天然适合构建 BST。为了保证平衡, 我们应该选择数组的中间元素作为根节点。”
- “这样, 根节点左侧的元素自然归属左子树, 右侧的元素归属右子树, 并且左右子树的元素数量尽可能平均, 有利于平衡。”

3. 递归函数设计:

- “定义一个递归函数 `build(left, right)`, 负责处理数组 `nums` 在 `[left, right]` 区间的部分。”

- “Base Case: 如果 $left > right$, 表示区间为空, 返回 `null`。”
 - “递归步骤:
 - 找到中间索引 `mid`。
 - 创建根节点 `root = TreeNode(nums[mid])`。
 - 递归调用 `root.left = build(left, mid - 1)` 构建左子树。
 - 递归调用 `root.right = build(mid + 1, right)` 构建右子树。
 - 返回 `root`。”
 - “主函数调用 `build(0, len(nums) - 1)`。”
4. **解释为何平衡:**
- “每次都取中间点, 确保了左右子树节点数量差最多为 1, 递归下去, 整棵树的高度差也会被限制在 1 以内, 满足高度平衡要求。”
5. **复杂度分析:**
- “时间复杂度 $O(N)$, 每个元素处理一次。”
 - “空间复杂度 $O(\log N)$, 递归栈深度, 因为树是平衡的。”
6. **细节与选择:**
- “计算 `mid` 时, 可以选择 $(left + right) // 2$ 或 $(left + right + 1) // 2$, 会产生结构略有不同的合法平衡 BST。”
 - “处理边界条件 $left > right$ 很重要。”